PROJET D'INFORMATIQUE CSC_42021

PI 4 : Convex hulls

11 février 2025

Emile Chaurang, Alexandre Paresy

TABLE DES MATIÈRES

1 Architecture et implémentation **2**

2 Algorithme de balayage **3**

3 Algorithme « diviser pour régner » **4**
3.1 Description de l'algorithme 4
3.2 Complexité temporelle 4

4 Résultats **5**

5 Visualisation **6**

INTRODUCTION

Étant donné un ensemble de points P, l'ensemble des parties convexes de l'espace contenant P est stable par intersection, ce qui implique l'existence d'une enveloppe convexe $\mathcal{C}(P)$ de P définie comme le plus petit convexe contenant P. De plus, en deux dimensions, lorsque P est fini, $\mathcal{C}(P)$ est un polygone convexe dont les sommets sont dans P, ce qui permet une représentation informatique pratique à utiliser. C'est dans ce cas que nous implémentions et analysons des algorithmes de recherche d'enveloppe convexe. Nous exhibons notamment deux algorithmes efficaces pour le calcul d'enveloppe convexe ainsi qu'un outil de visualisation de ces enveloppes convexes.

1 ARCHITECTURE ET IMPLÉMENTATION

La plus grande partie du code sera écrite en C++. Ce choix a été effectué en raison de la vitesse d'exécution de C++ qui est beaucoup plus rapide que des celle de langages plus haut niveau comme Python, ainsi que du fait que les deux membres du groupe sont familiers avec la programmation dans ce langage.

Afin de ne pas surcharger notre projet d'un nombre trop grand de fichiers, nous avons décidé d'écrire une grande partie du projet dans un seul fichier. Ainsi, `convex_hull.cpp` contient l'essentiel du code demandé dans l'énoncé et se décompose en 3 parties. La première partie concerne la génération des datasets de l'énoncé, avec `vector<point> genX(int n)` où `X` $\in$ `{A,B,C,D}` et `point` est une structure qui contient l'abscisse `x` et l'ordonnée `y` d'un point. Ces fonctions renvoient non seulement les ensembles voulus mais les écrivent également dans des fichiers `datasetX.txt` pour d'autres usages ultérieurs. Nous nous sommes assurés d'écrire des fonctions de complexité en moyenne $O(n)$, ce qui nous permet de générer aisément des datasets de taille 10^7 en moins d'une seconde. Nous nous servons notamment d'un générateur aléatoire efficace et rapide de la librairie standard de C++, `mt19937` pour Mersenne Twister 19937, ainsi que de la fonction standard `shuffle` dont le langage garantit une complexité en $O(n * v)$, où v est la complexité de notre générateur aléatoire de nombre, qui est ici en $O(1)$.

La deuxième partie de ce fichier est dédiée à l'algorithme de balayage, que nous détaillons en partie 2 et la troisième partie à l'algorithme diviser pour régner, que nous détaillons en partie 3. Le fichier `convex_hull.cpp` est également accompagné de son header `convex_hull.h` ainsi que d'un `Makefile` et d'un `readme.md` qui contient les commandes de compilation.

Pour ce qui est des outils de visualisation, nous avons un fichier `visualization.py` qui permet d'obtenir une visualisation des enveloppes convexes de chacun des 4 datasets. Nous avons également réalisé un visualiseur en direct dans `visu_hull.cpp` qui utilise la librairie graphique SFML de C++ et qui est généralisable à toutes sortes d'algorithmes de géométrie.

Nous nous assurons également d'avoir une précision suffisante en utilisant le type `long double` dont la mantisse fait 64 bits sous une architecture x86. Ainsi, puisque nous utilisons seulement des opérations de comparaison et des opérations arithmétiques, nous pouvons garantir une précision au moins aussi bonne qu'à 10^{-12} près.

2 ALGORITHME DE BALAYAGE

L'idée du balayage est la suivante : nous pouvons décomposer $\mathcal{C}(P)$ en une enveloppe supérieure $\mathcal{C}(P)_u$ et une enveloppe inférieure $\mathcal{C}(P)_d$ que l'on relie au niveau des points de plus petite et plus grande abscisse (nous supposons ici que ces points sont uniques).

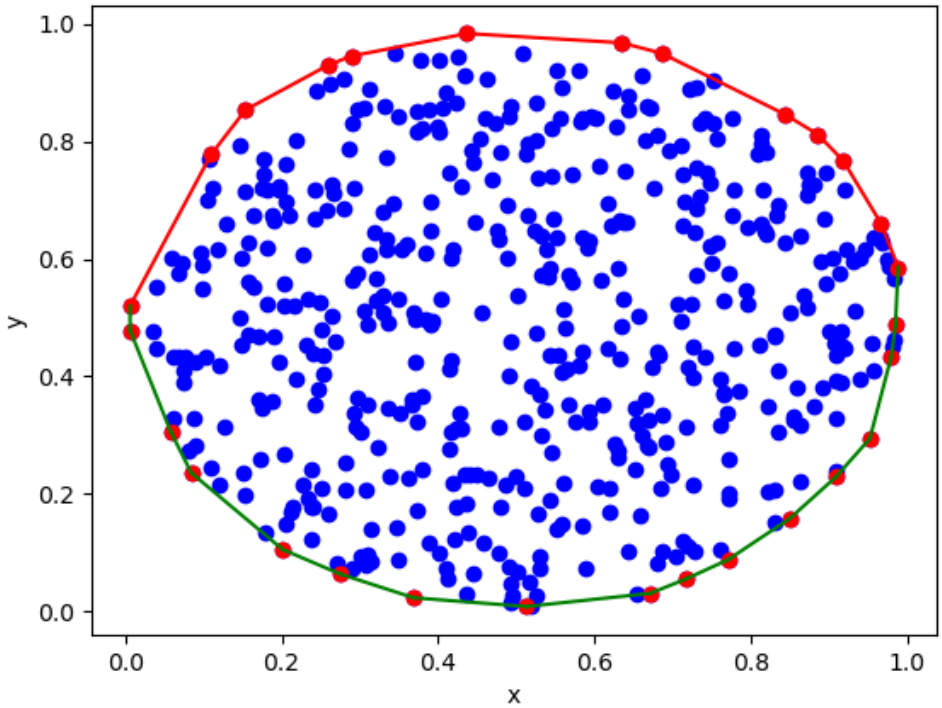

Figure 1 – Décomposition de l'enveloppe supérieure $\mathcal{C}(P)_u$ (rouge) et inférieure $\mathcal{C}(P)_d$ (vert)

La construction des deux moitiés de l'enveloppe est symétrique donc nous nous intéressons seulement à la construction de $\mathcal{C}(P)_u$. Voici le pseudocode correspondant :

```
Fonction EnveloppeSup(points):
    Trier points par ordre croissant d'abscisse
    st <- pile vide
    Pour chaque point x dans points :
        Si st est de taille <= 1:
            Pousser x dans st
```

```
Sinon:
    Soient a et b les deux points au sommet de st
    Tant que a et b sont tels que l'angle orienté (ax,bx) est positif:
        retirer b du sommet de st
        Actualiser a et b, sortir de la boucle s'ils n'existent pas
    Pousser x dans st
```

Ainsi, l'algorithme peut être implémenté avec une simple structure de vecteur pour la liste de points et de pile. Nous implémentons une fonction `orient` qui prend trois points `a`, `b` et `c` et vérifie si l'angle entre `ac` et `bc` est orienté positivement ou négativement avec un simple produit en croix et nous trions notre liste de points avec la fonction standard `sort` qui trie sur la première coordonnée en complexité $O(nlog(n))$. Nous proposons une visualisation et une analyse des résultats en partie 4.

Notons qu'il existe également une variante de cet algorithme, le parcours de Graham, où les points ne sont balayés non plus par ordre d'abscisse, mais par ordre d'angle par rapport à un pivot (qui est souvent choisi comme le point d'abscisse le plus faible). La ligne de balayage ne fait donc non plus une translation mais une rotation, mais le principe du reste de l'algorithme est exactement le même.

3 ALGORITHME « DIVISER POUR RÉGNER »

3.1 Description de l'algorithme

Le deuxième algorithme auquel nous nous sommes intéressés repose sur une division du problème selon un axe vertical dans une approche proche du paradigme « diviser pour régner ». L'algorithme détermine comme le précédent les enveloppes supérieures et inférieures puis les combine pour obtenir l'enveloppe convexe $\mathcal{C}(P)$.

Remarquons en premier lieu que l'ensemble des segments de l'enveloppe supérieure est exactement l'ensemble S des segments de sommets de P dont la droite est située au-dessus de tous les points de P. En effet, si une droite passant par deux points A et B de P est située au-dessus de tous les points de P, alors $[AB]$ est dans l'enveloppe supérieure par convexité. Réciproquement, les segments du bord supérieur de $\mathcal{C}(P)$ sont des segments $[AB]$ avec $\{A, B\} \subset P$ donc tout point de $\mathcal{C}(P)$ est situé en-dessous d'un tel segment $[AB]$, et par convexité, tout point de P est en-dessous de la droite (AB). On a donc l'égalité entre S et l'ensemble des segments de l'enveloppe supérieure de P. Il s'ensuit que déterminer cette enveloppe supérieure revient à déterminer ces segments.

L'algorithme utilise une procédure permettant, étant donné $Q \subset P$ et $x_m \in \mathbb{R}$, de trouver les points A et B de P tels que (AB) soit au-dessus de tous les points de Q et minimisant l'ordonnée de l'intersection entre (AB) et la droite D d'équation $x = x_m$. La paire $\{A, B\}$ est appelée la base. A et B sont situés de part et d'autre de la droite D. La procédure consiste à choisir en premier lieu pour A et B deux points quelconques de part et d'autre de D, puis à itérer sur les points restants et à mettre à jour la base dans le cas où le point considéré est au-dessus de (AB).

Une fois cette procédure implémentée, on peut résoudre le problème global en considérant x_m une médiane des abscisses des points de P puis en appliquant la procédure précédente à l'abscisse x_m pour obtenir deux points A et B. Il s'ensuit que les abscisses des sommets des segments restants de S ne sont pas situées entre les abscisses de A et de B. Il suffit alors d'appliquer l'algorithme récursivement sur les deux parties de P dont les abscisses ne sont pas entre celles de A et de B.

3.2 Complexité temporelle

Sans compter les appels récursifs, la procédure de recherche d'une base s'exécute en temps linéaire en le nombre de points grâce à une recherche de médiane avec l'algorithme quickselect avec pivot implémenté par

l'algorithme de la médiane des médianes. La complexité totale est $\mathcal{O}(nlogn)$ avec n le nombre de points. Un intérêt de cet algorithme par rapport au précédent est donc qu'il permet de conclure en temps linéaire en n lorsque h est borné, c'est-à-dire que l'on résout efficacement les cas les plus triviaux.

4 RÉSULTATS

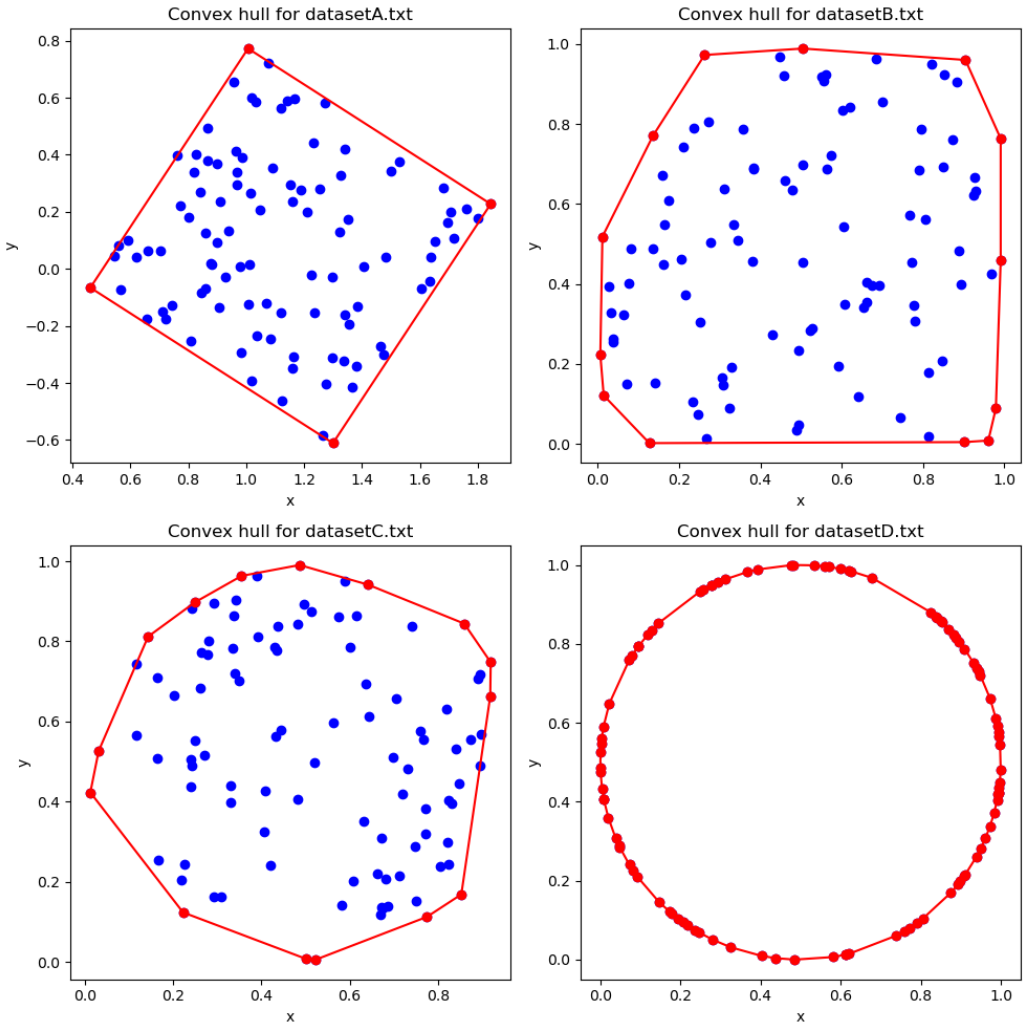

Figure 2 – Enveloppe convexe renvoyée par les deux algorithmes sur chacun des 4 datasets pour $N = 200$

Nous avons implémenté des outils permettant de mesurer les performances et la cohérence des résultats des algorithmes (voir le fichier README). Nous avons testé nos algorithmes sur des cas générés aléatoirement et nous les avons comparés. Nous avons trouvé que les résultats étaient bien les mêmes pour nos deux algorithmes pour chacun des cas rencontrés. Nous trouvons également que les performances du deuxième algorithme sont grandement améliorées lorsque l'enveloppe convexe est un polygone avec peu de côtés, ce qui est cohérent avec la théorie.

Algorithme	Balayage	DNC
N = 100	6.3e-5s	0.004s
N = 1000	0.0005s	0.13s
N = 10000	0.005s	1.4s

Figure 3 – Temps d'exécution moyenné sur les 4 datasets et 5 exécutions

5 VISUALISATION

Nous avons implémenté un outil de visualisation des algorithmes implémentés avec la bibliothèque SFML en C++, en plus d'un algorithme plus simple en Python utilisant Matplotlib. L'intérêt de l'outil C++ est qu'en plus d'afficher le résultat final, il permet de voir les différentes étapes de l'algorithme de balayage en direct. Cela nous donne également une petite partie d'informatique graphique. L'outil en Python récupère directement le contenu des fichiers .txt pour tracer les enveloppes convexes sur les nuages de points des datasets.